IoTVerif: An Automated Tool to Verify SSL/TLS Certificate Validation in Android MQTT Client Applications

Khalid Alghamdi, Ali Alqazzaz, Anyi Liu, Hua Ming
Oakland University
Rochester, MI, USA
{kaalgham,aalqazzaz,anyiliu,ming}@oakland.edu

ABSTRACT

Developing secure Internet of Things (IoT) applications that are free of vulnerabilities and resilient against exploit is desirable for software developers and testers. In this paper, we present IoTVerif, an automated tool that can verify SSL/TLS (Secure Socket Layer/Transport Layer Security) X.509 certificate validation of IoT messaging protocols utilized by real-world IoT client applications. IoTVerif does not require any prior knowledge about the messaging protocol, but simply correlates the observed network trace of an application with its execution context. IoTVerif helps IoT client application developers identify the SSL/TLS vulnerabilities based on certificate validation. We specifically target MQTT, a broker-based protocol that has attracted increasing popularity in the IoT application market.

We used IoTVerif to analyze the server X.509 certificate validation in 15 well-known MQTT client applications. Our result revealed that 5 (33.3%) of the applications examined are vulnerable to man-in-the-middle (MITM) and/or TLS renegotiation attacks. Our result also shows that IoTVerif can generate a Finite State Machine (FSM) that depicts the interaction between the application and the IoT broker and automatically identifies various attacks. It has the potential to reverse-engineer the emerging IoT messaging protocols and identify the vulnerabilities in the IoT applications.

CCS CONCEPTS

• **Security and privacy** → **Mobile and wireless security**; • **Networks** → *Protocol testing and verification*;

KEYWORDS

MQTT protocol; Model checking; SSL/TLS; Certificate validation

ACM Reference format:
Khalid Alghamdi, Ali Alqazzaz, Anyi Liu, Hua Ming. 2018. IoTVerif: An Automated Tool to Verify SSL/TLS Certificate Validation in Android MQTT Client Applications. In *Proceedings of Eighth ACM Conference on Data and Application Security and Privacy, Tempe, AZ, USA, March 19–21, 2018 (CODASPY '18),* 8 pages.
https://doi.org/10.1145/3176258.3176334

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CODASPY '18, March 19–21, 2018, Tempe, AZ, USA
© 2018 Copyright held by the owner/author(s). Publication rights licensed to Association for Computing Machinery.
ACM ISBN 978-1-4503-5632-9/18/03…$15.00
https://doi.org/10.1145/3176258.3176334

1 INTRODUCTION

The importance of IoT security is widely understood and agreed upon [16]. SSL/TLS protocols are the most commonly used mechanisms for ensuring the security and privacy of the network communication and the user data integrity from MITM [23] attack. They have gone through a lot of upgrading since they were first introduced, for instance, SSLv1.0, SSLv2.0, SSLv30, and TLSv1.0. SSL/TLS security depends on the validation of the server X.509 certificate in the MQTT client application. The server sends the X.509 certificate to the client during the TLS handshake phase, which is responsible for authentication and exchanging certificates between the MQTT client and the broker for necessary secure sessions. Certificate validation depends on verifying the certificate chain, subjectCN (Common Name), signature, and certificate validity.

In this paper, we present IoTVerif, an automated tool that aims at verifying SSL/TLS X.509 certificate validation of IoT messaging protocols utilized by real-world IoT MQTT client applications and thus identifying the vulnerabilities in the applications. IoTVerif is based on the key observation that the network traffic of an application is *highly correlatable* with the execution context of an IoT applications (e.g., the API calls). By monitoring the execution of applications and their network trace, a finite state machine (FSM), which depicts the interaction between the application and the message broker via the protocol, is generated and then streamlined to a formal tool to be verified. Because IoTVerif learns the inbound and outbound messages and infers the states of the protocol, it does not require any prior knowledge about the protocol. We have tested IoTVerif with real-world applications that use the popular protocol, namely MQTT. We show that our tool can generate the FSM accurately. Furthermore, it leverages the general model checking tool to automatically identify various attacks, such as MITM and TLS Renegotiation.

The reminder of this paper is organized as follows. Section 2 briefly describes the MQTT protocol and other techniques used in this paper. Section 3 describes the scope of the problem and defines the terminologies. Section 4 presents the system architecture of IoTVerif and details its working mechanism. Section 5 evaluates the effectiveness of IoTVerif. Section 6 covers the related work. Section 7 concludes the paper.

2 BACKGROUND

Our security verification focuses on verifying the validation of SSL/TLS certification of Android IoT applications utlizing MQTT protocol. These applications used Eclipse Paho project. To perform the task of verifying these MQTT clients applications, we need to have a deep understanding of the MQTT protocol, SSL/TLS protocol, and the tools used in this work.

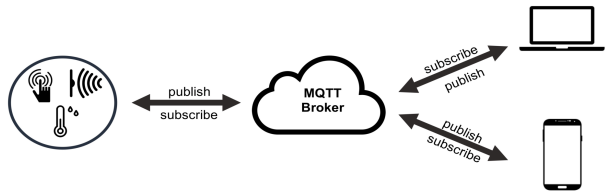


Figure 1: MQTT architecture for publishing and subscribing

2.1 MQTT

MQTT is a asynchronous, lightweight, and based on publish-subscribe architecture protocol. According to the way messages are delivered, MQTT messaging protocols can be categorized as *broker-based* protocol [2] and designed for machine-to-machine (M2M) communication. The broker controls the distribution of the information as illustrated in Fig. 1. It stores, forwards, filters, and prioritizes publish requests from the *publisher* (the message producer) to the *subscriber* (the message consumer). Clients switch the role between publisher and subscriber depending on their objectives. MQTT provides Three levels of quality of service (QoS), QoS 0, 1, and 2. The QoS is an agreement between the message sender and receiver regarding the guarantees of delivering the message [18].

According to a recent survey [16], MQTT protocol has been ranked the most popular broker-based protocol among IoT messaging protocols, which demonstrates its advantages for interacting with resource-constrained IoT devices over low bandwidth and unreliable environments.

2.2 SSL/TLS & Android

The SSL and its successor TLS are cryptographic protocols. The fundamental goal of SSL/TLS is to protect network communication and data integrity from tampering. To start a secure connection, the client must securely obtain the server X.509 certificate and extract the server public key, which must be signed by a Certificate Authority (CA). When the client receives the X.509 certificate from the server, the client must check and validate the certificate components [9]. The certificate basic validation includes a) validating that the subject (CN) of X.509 certificate and the URL matches; b) validating that the certificate is signed by the trusted CA; c) validating that the signature is correct; and d) validating that the certificate is not expired. Furthermore, most of the messaging providers of MQTT, such as HiveMQ [14] and Microsoft Azure [5], recommend MQTT client developers use the highest TLS version and not use SSLv3 or any prior versions. The reason is that all SSL versions are considered broken and vulnerable to well-known attacks, while TLSv1.0 is exposed to TLS Renegotiation attack.

Android SDK 4.0 and later provides several packages to establish the connection and access the network, such as `java.net`, `javax.net`, `android.net` or `org.apache.http`. Android allows developers to customize the SSL/TLS environment. However, if the SSL/TLS is not correctly implemented, the user sensitive data may leak via vulnerable SSL. Fahl *et al.* [12] describe SS/TLS (mis-)use cases, such as trusting all certificates and allowing all host names, of such customization that could cause a broken SSL channel.

3 PROBLEM SCOPE & TERMINOLOGIES

The primary goal of IoTVerif is to generate a finite state machine that depicts the interaction between an IoT application and the message broker, verifies the security properties with respect to the usage of SSL/TLS and identifies the vulnerabilities in IoT applications. In particular, we analyzed the applications' vulnerabilities against *TLS Renegotiation* [21] and MITM attacks. The *key* observation is that the Android application level API invocations essentially lead to communication traffic that can be monitored at the network layer, and vice versa.

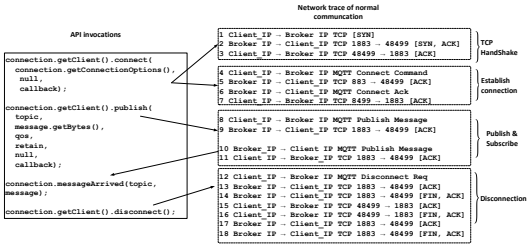
3.1 Motivating Example

To understand our problem better, we illustrate these concepts with an actual attacking scenario that leverages the vulnerability of MQTT applications and launches an *MITM* attack through the Android application, `MQTT Viewer (IoT Client)`. It is a simple client-side application that controls various IoT sensors, such as reading from humidity sensors and sound sensors, and turns on/off LED lights through Wi-Fi. Fig. 2 shows the code snippet of `MyMQTT` and the network traffic between the application and MQTT broker, the major responsibility of which is to exchange data between the application and IoT sensors through the broker through publishing and subscribing. The publishing- and subscribing-related *Paho* API calls are highlighted. For simplicity, we omit the functionalities of the application, such as data processing, and their corresponding network traffic. In addition, we abstract away some details such as the class definition, and consolidate relevant application-side API calls.

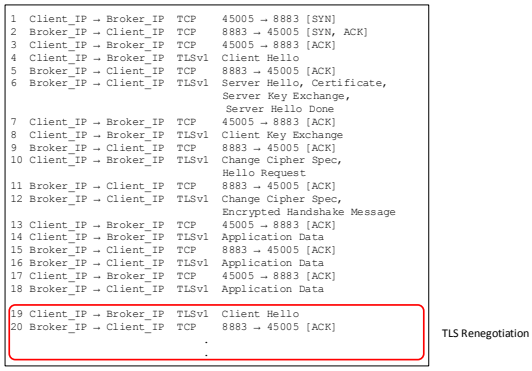
As illustrated in Fig. 2(a), a normal session between the application and MQTT broker always starts with a client-side API invocation `connect` that triggers the TCP handshake (packets 1 - 3) followed by MQTT connection (packets 4 - 7). Then, the invocations of `publish` cause the traffic that publishes the data with the broker (packets 8 and 9). After that, the network traffic (packets 10 and 11) cause the API invocation of `messageArrived`. Finally, the invocations of `disconnect` terminates the session with the broker (packets 12-18). Fig. 2(b) shows a *TLS Renegotiation* attack [21] on the same application, which use an TLS port. The invocation of `connect` triggers the TCP handshake (packets 1 - 3) followed by TLS handshake (packets 4 - 13). TLS handshake allows the application and broker to authenticate each other and negotiate the encryption algorithm and cryptographic keys. Then the application and broker exchange data (packet 14 - 18). The circled network packets (packets 19 and 20) show the beginning of the TLS renegotiation handshake. The cause of the second TLS handshake is that we inject plain text in packet 18 payload to force the client to renegotiate the TLS handshake with the broker. Then MITM attacker blocks *Hello Client* packet and form a new *Hello Client* using the client IP address. When the attacker finishes communicating with the broker, the blocked *Hello Client* will be released and sent to the broker. We performed the attack by creating our own Ruby module atop of BetterCap [6] tool.

We notice some security issues of MQTT client applications caused by either the developers' carelessness or malicious intention. For example, some applications are still using old versions of SSL/TLS, such as SSLv3 and TLSv1.0 which are known to be broken nowadays and vulnerable to various attacks [22]. TLSv1.1

Figure 2: MQTT-related Paha API calls and Network Trace



(a) MQTT-related Paha API calls and Network Trace of Normal Communication Over the Default Port (1883)



(b) Network Trace of Anomalous Communication Over Secure Port (8883)

and TLSv1.2 solve the security issue of renegotiation by adding a *renegotiation_info* extension to the *Client Hello* and *Server Hello* packets. If the extension is not present, the server does not support secure renegotiation [21]. Although most applications provide secure communication over TLSv1.1 or TLSv1.2, some do not validate the server certificate components as described in Section 2.2. Such checking prevents MITM attacks by ensuring the certificate was not changed during the transmission. If the application does not provide certificate checking, the attacker can replace the certificate during the transmission with his own and pretend to be the broker.

As mentioned, the primary goal of IoTVerif is to verify the security properties with respect to use of SSL/TLS by correlating the observed network traffic of an application with its execution context. From the network's perspective, we can easily identify the SSL/TLS version as illustrated in Fig. 2(b), which is too difficult to obtain from the application layer by using code instrumentation. However, the network traffic does not provide any information about whether the application validates the X.509 certificate components. Thus, to check the application certificate validation, we need to analyze and instrument the application layer. Therefore, we consider both network traffic and execution context layers to generate the finite state machine and verify the SSL/TLS validation of the application.

3.2 Terminologies

Let us consider the interaction between an application and the message broker. In particular, we focus on those APIs that directly engage broker-based IoT protocol activities such as establishing connection, publishing, subscribing, and disconnecting the connection. We use $A = \{\alpha_1, \alpha_2, ..., \alpha_i\}$ to represent those APIs. The execution state of an application is closely related to the particular order of API invocations and their context. We extract the execution states using the technique of code instrumentation as described in Section 4.2. As a result, we propose the execution state of Android applications with regard to their respective API invocations, and define it as a four-tuple $S = \langle \Phi, M, I, R \rangle$, where Φ denotes the method that encompasses an API call, M denotes the certificate subjectCN validation, I denotes the certificate signature validation, and R denotes the certificate expiration.

From the network's perspective, we use $P = \{p_1, p_2, ..., p_j\}$ to represent the set of MQTT-based network packets. Correspondingly, an IoT connection session is denoted as $CS = \langle p_1, p_2, ..., p_l \rangle$, which is comprised of all the network packets in that session. A network packets fragment with the length of k is denoted as $NPF_{i,k} = \langle p_i, ..., p_{i+k} \rangle$, where $1 \leq i \leq l$. Based on these definitions, we formulate the finite state machine that depicts the interaction between the application and the message broker as MQTT-FSM, which is a five-tuple $\mathcal{F} = \langle Q, \Sigma, \delta, I, F \rangle$, where:

- Q is a finite set of states S;
- Σ is the set of session fragments $S_{i,k}$;
- $\delta \subseteq Q \times \Sigma \times Q$ is a set of transitions;
- $I \in Q$ is the initial state; and
- $F \in Q$ is the final state.

4 SYSTEM DESIGN

4.1 System Architecture

The system architecture of IoTVerif is illustrated in Fig. 3 and is comprised of four main components shown in shaded boxes. There are in total seven major steps to verify the security properties of MQTT application. The *Instrumentation Engine* takes the application (.apk) file as an input and statically analyzes the application and identifies and instruments the APIs that are directly involved in IoT-related functionalities, such as establishing connecting, publishing, subscribing, and ending connection (**step ❶**). It also identifies whether the application validates the broker X.509 certificate or not. Then IoTVerif launches the emulator, installs, and starts the instrumented application (**step ❷**). After that, a special Android application installed in the emulator, namely, Android tcpdump, sniffs the network traffic between the application and the broker (**step ❸**) and forwards it to the *Network Traffic Parser*. The *Network Traffic Parser* parses the traffic and identifies the SSL/TLS version and other network traffic related to communication between the client and broker. Using the result of the *Network Traffic Parser* (**step ❹**) and the *Instrumentation Engine* (**step ❺**), the *FSM Generator* generates MQTT-FSM and forwards it to the *Symbolic Model Checker* (**step ❻**), which takes security specifications and determines whether the IoT application is vulnerable to certain security exploits such as MITM and/or TLS Renegotiation attack (**step ❼**). The following subsections present the technical details of IoTVerif.

4.2 Instrumenting IoT App

To generate and verify the correct model, we need to properly answer at least three key questions: 1) how to *identify* all the relevant

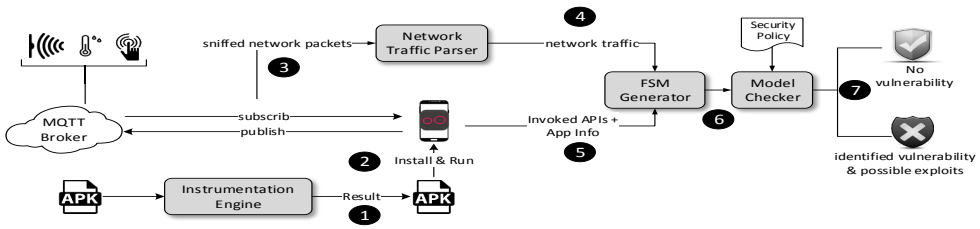


Figure 3: The system architecture of IoTVerif.

APIs with invocations that directly trigger the IoT-related network traces?; 2) how to *infer* the states of an application during its execution?; and 3) how to *correlate* these APIs with the particular network packets in a trace?

To answer the first two questions, we leverage the technique of code instrumentation, which has been widely used in Android application testing and security [3], and has demonstrated its advantages of enhancing testing with no knowledge of the code. In particular, because the Android SDK provides a set of IoT-related packages (e.g., Paho APIs), our instrumentation engine covers the well-defined IoT API calls. Based on their specification as well as our manual analysis with a number of IoT applications, we have obtained 4 commonly used IoT-related Paho APIs, *paho.android.service.MqttService.connect*, *paho.android.service.MqttService.publish*, *paho.client.mqttv3.internal.CommsCallback.messageArrived*, and *paho.android.service.MqttService.disconnect*. We found that most applications directly use them to facilitate IoT-related functionalities. In addition, the *instrumentation engine* instruments the server certificate validation methods in the application, for instance, `verify`, and `checkValidity` methods from *javax.security* library. These methods validate the certificate subject distinguish name (DN), and the certificate expiration, respectively.

To answer the third question, it is important to collect a substantial amount of network trace packets that achieve high code coverage for the IoT functionalities. To do that, we developed a simple and fully automated tool based on Monkey [25] to generate the network traces automatically. The manual effort is to provide the configuration settings for connecting the application with the broker. Specifically, we invoke the `am` command provided by Monkey to run the application and stop executing it after 60 seconds. To correlate the APIs α_i and the packet p_j, we define two heuristic relationships, namely, *prepare-for* and *lead-to*. The relationship *prepare-for* states that α_i triggers p_j. It can be observed in Fig. 2(a), when the invocation of *publish* triggers a segment of the network traffic (packet 8). The relationship *lead-to* states that α_i is triggered by p_j and can be observed in Fig. 2(a) when the invocation of *messageArrived* is triggered by a segment of the network traffic (packet 10).

We define the relationships of *prepare-for* and *lead-to* as follows:

- $\alpha_i \overset{prepare-for}{\vdash} p_j$: $p_j.time - \alpha_i.time < \gamma$ and $\alpha_i.time \leq p_j.time \leq \alpha_{i+1}.time$
- $\alpha_i \overset{lead-to}{\vdash} p_j$: $\alpha_i.time - p_j.time < \gamma$ and $\alpha_{i-1}.time \leq p_j.time \leq \alpha_i.time$

where $\alpha_i.time$ and $p_j.time$ refer to the time stamps of the API calls and the packet, respectively; while γ is a user-defined parameter for the timing window size. Because the mapping between α_i and p_j is one-to-many, a total number of k packets that satisfies the criteria *prepare-for* α_i or *lead-to* α_i forms the session segment $S_{j,k}$.

Algorithm 1 Constructing MQTT-FSM

Inputs:
A network trace P and the set of states $\mathbb{S}$.
Output:
The *MQTT-FSM* as a five-tuple, $\mathcal{F} = \langle Q, \Sigma, \delta, I, F \rangle$.
Initialize:
$Q = \phi; \Sigma = \{\varepsilon\}; \delta = \phi; I = \phi; F = \phi;$
$pre = NUL; post = NUL; e = \phi; i = 0;$
loop:
for *next* $p_i \in P$
if observe a $S_i \in \mathbb{S}$ && $S_i \notin Q$ **then**
  $Q = Q \cup \{S_i\}$;
  /*Identifying different transitions*/
  if $S_i.I$ == `disconnect` **then**
    $pre = S_i$; $F = post = S_F$;
  else
    $pre = S_i$; $post = look_ahead(pre)$;
  end if
  /*Combining session fragments*/
  Let NPF_1 and NPF_2 be two empty lists;
  $len1 = 0; len2 = 0;$
  if $\forall p_i \in P : \alpha_i \overset{prepare-for}{\vdash} p_i$ **then**
    $NPF_1 = append(NPF_i, p_i); len1{+}{+};$
  end if
  if $\forall p_j \in P : \alpha_{i+1} \overset{lead-to}{\vdash} p_j$ **then**
    $NPF_2 = append(NPF_i, p_i); len2{+}{+};$
  end if
  /*Updating alphabet set*/
  $e = concat(NPF_{i,len1}, NPF_{j,len2});$
  $\Sigma = \Sigma \cup \{e\};$
  /*Updating transition set*/
  $\delta = \delta \cup \{((pre, e), post)\};$
  $i{+}{+};$
end if
end loop:
Return $\langle Q, \Sigma, \delta, I, F \rangle$;

4.3 Generating MQTT-FSM

In essence, the key to constructing an FSM that depicts interaction between an IoT application and message broker is determining the states of the interaction. As we mentioned, we use a four-tuple S to approximate the states of the interaction, which is generated by the component of model generator.

Algorithm 1 describes the process of constructing the MQTT-FSM with the information of network trace and application state. Before running the algorithm, we first define a function, namely, *look_ahead*, which is looks ahead of the string and returns the next states. This function is similar to the *lookahead* function that is widely adopted in compiler-related techniques [13]. Two string functions, namely, *append* and *concat*, are also used. We further define a hypersthenic final state S_F. The algorithm treats the strings of the network traffic P and the API calls $\mathbb{S}$ as the input and the five-tuple $\langle Q, \Sigma, \delta, I, F \rangle$, i.e., the MQTT-FSM, as the output. First, it scans the packets and API calls and determines different transitions (lines 9 - 13). Then it identifies the network packets fragments NPF_1 and NPF_2 for two adjacent API calls, namely, α_i and α_{i+1}, respectively (lines 17 - 19 and lines 20 - 22). After that, it concatenates NPF_1 and NPF_2 as a new symbol e and includes it in the alphabet (lines 24 and 25). It also includes the transition $\{((pre, e), post)\}$ into the transition set Σ (line 27). Finally, it returns the constructed MQTT-FSM as a five-tuple.

Table 1: General CTL formulas

1	$\mathcal{M}, S \models AG(R = connect \rightarrow AX(R = publish \vee R = messagArrived \vee R = disconnect))$
2	$\mathcal{M}, S \models AG(R = connect \rightarrow AX(NT = e1e2 \vee NT = e1e2e4))$
3	$\mathcal{M}, S \models AG(R = publish \rightarrow AX(R = publish \vee R = messageArrived \vee R = disconnect))$
4	$\mathcal{M}, S \models AG((NT = e1e2e4 \vee NT = e3e4 \vee NT = e3) \rightarrow AX(R = messageArrived))$
5	$\mathcal{M}, S \models AG(R = publish \rightarrow AX(NT = e3 \vee NT = e3e4))$

Table 2: Security CTL formulas

1	$\mathcal{M}, S \models AG(TLS = TRUE)$
2	$\mathcal{M}, S \models AG(TLSv >= 1)$
3	$\mathcal{M}, S \models AG(certCheckCN = TRUE)$
4	$\mathcal{M}, S \models AG(SignedCA = TRUE)$
5	$\mathcal{M}, S \models AG(checkSigCorr = TRUE)$
6	$\mathcal{M}, S \models AG(checkCAEXP = TRUE)$
7	$\mathcal{M}, S \models AG(\neg NT = RST \quad AU \quad R = disconnect)$
8	$\mathcal{M}, S \models AG(R = \texttt{connect} \rightarrow AF(R = \texttt{connect} \quad AU \quad R = \texttt{disconnect}))$
9	$\mathcal{M}, S \models AG(NT = e2 \rightarrow (TLS \wedge TLSv >= 1 \wedge SignedCA \wedge certCheckCN \wedge checkCAEXP))$

4.4 MQTT Security Properties

After MQTT-FSM is generated, the component of symbolic model checker converts it to a symbolic model and verifies the security properties for the model.

The properties are expressed in CTL [11]. Such properties can be used to check whether the MQTT application is vulnerable to security attacks, such as MITM and TLS Renegotiation attacks. We developed 14 CTL formulas based on the existence of the SSL/TLS certificate validation in the client applications and the interaction between the application and the network. The CTL formulas are divided into 2 categories. The first category verifies general properties such as the application API execution order and the network behavior, while the second category verifies security properties such as the existence of the TLS, the TLS version, the certificate validation, and the connection integrity.

Table 1 shows the general CTL formulas. In all formulas, $\mathcal{M}$ and S represent MQTT-FSM and the current state of MQTT-FSM, respectively. R is the variable that represents the API call and NT is used to denote the network traffic fragment. In the following, we explain the CTL formulas in Table 1. The first formula holds for all paths and future states, the `connect` API call will be followed by either `publish` or `messageArrived` or `disconnect` API call. Similarly with the third formula, the `publish` API call will be followed by either `publish`, `messageArrived`, or `disconnect` API call. The second formula holds for all paths and future states, the `connect` API call will be followed by network traffic `e1e2` or `e1e2e4`. Similarly with the fifth formula, the `publish` API call will be followed by network traffic `e3` or `e3e4`. The fourth formula holds for all paths and states, the network traffic `e1e2e4` or `e3e4` or `e4` will trigger `messageArrived` API in the application.

Table 2 shows the security CTL formulas. These formulas use notations similar to the general CTL formulas. The first and second formulas hold for all paths and all states if the application provides TLS connection and the application use the latest TLS version respectively. The next four formulas hold for all paths and all states if the application validates the certificate subjectCN, certificate signature, signature correctness, and the validity of the certificate. The seventh formula, which verifies the connection integrity, holds for all paths and all states, the reset (`RST`) network packet will not appear *until* disconnect API call appears. The TCP `RST` flag is used to terminate the connection immediately, mostly because of a fatal error in the communication. The eighth formula holds for all paths and all future states, there is no second `connect` API call until `disconnect` API call appears. The last formula checks the TLS renegotiation handshake. It holds for all paths and future states, the TLS renegotiation network packet *e2* occurs over TLSv1.1 or TLSv1.2, and the application validates the server X.509 certificate.

5 EVALUATION

5.1 Experimental Setup

To evaluate the effectiveness of IoTVerif, we use real-world Android IoT applications as illustrated in Table 3. To obtain the appropriate apps for our evaluation, we first use Google Play Crawler [1] to crawl mobile applications from Google Play, and then only keep those with installation counts greater than 1000 and that were developed with MQTT Paho API. As a result, we eventually had 15 apps to test. The API calls are triggered by Monkey and executed inside Genymotion Emulator. The instrumentation engine of IoTVerif is developed based on FlowDroid [4]. The model generator is implemented by Java. The symbolic model check leverages *NuSMV* [8].

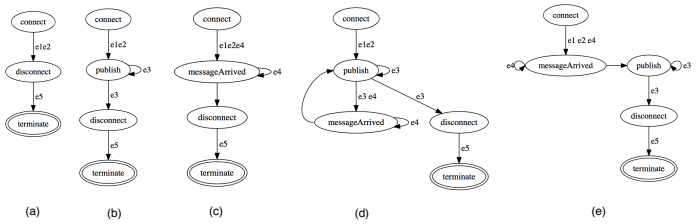


Figure 4: MQTT-FSM, which models both sides of application and network of Android application, namely, `MQTT Dashboard`.

Table 4: The Notations of MQTT-FSM Transitions

Notation	Structure of Traffic
e1	SYN(A, B)→SYN,ACK(B, A)→ACK(A, B)
e2	ClientHello(A, B)→ ServerHello, Certificate, ServerKeyExchange, ServerHelloDone(B, A)→ ACK(A, B)→ ClientKeyExchange(A,B)→ACK(B,A) → ChangeCipherSpec, HelloRequest(A,B) → ACK(B,A) → ChangeCipherSpec, EncryptedHandshakeMessage(B,A) → ACK(A,B)
e3	ApplicationData (A, B)→ACK (B, A)
e4	ApplicationData (B, A)→ACK (A, B)
e5	ApplicationData (A, B) →ACK (B, A)→FIN,ACK (B, A)→ FIN,ACK (A, B) → ACK (B, A)→ACK (A, B)
NOTE : A = app , B = broker	

Table 3: The category, installs, app Name, version, size, and total number of activities for the 15 tested apps

Category	#install	App Package Name	Version	App size	Total # of Activities
Comm	50,000	net.routix.mqttdash	4.4	5.1 M	12
Tools	50,000	com.deepeshc.mqttrec	2.4	480 K	5
Tools	50,000	at.tripwire.mqtt.client	1.0	1.1 M	2
Tools	50,000	ru.esp8266.iotmanager	1.5.5	8.3 M	3
Tools	50,000	com.thn.iotmqttdashboard	1.9.3	3.3 M	20
Tools	10,000	net.nosybore.mqttpublishplugin	1.4.0	1.2 M	1
L&D	5,000	com.codifythings.universalmqttclient	1.1	5 M	7
Tools	5,000	com.vinmacro.vinmacromqtt	1.0	3.6 M	5
Tools	5,000	info.laptrinhpic.easycontrol.mqttplugin	1.0.2	1.2 M	2
Tools	5,000	net.sabamiso.android.simplemqttviewer	1.3	100.9 K	2
Tools	1,000	br.com.bintechnology.mqttclient	0.4.1	3.5 M	2
Comm	1,000	com.patterns.io.mqttpatterns	1.0	4.3 M	7
Tools	1,000	com.cmmakerclub.iot.cmmciotswitch	1.0	1.7 M	4
Tools	1,000	in.dc297.mqttclpro	2.3.1	2.7 M	9
Tools	1,000	com.hanzycanada.mqttspeed	0.05	7.5 M	8

The IoT platform is constructed on both Arduino Yun and Raspberry Pi, which comprises the sensors for temperature, humidity, LED, metal touch, and sound.

5.2 The Construction of MQTT-FSM

The design of Android applications is based on event-driven, depends on asynchronous programming [26], and is executed by a single event-loop thread [17]. Event-driven programming relies on the user activities and events, such as button presses, touch screens, and receiving network packets. During the running of Android applications, these events are delivered to the applications in non-deterministic order [7]. As a result, IoTVerif generates non-deterministic state machines based on the execution order of the events and the network traffic.

Fig.4 illustrates the generated MQTT-FSM, which models the benign activities on both sides of application and network of `MQTT Dashboard` Android application. For simplicity, we only keep the API names of each state. The network traffic fragments as the elements of the alphabet are shown in Table 4.

As Fig.4(b) shows, it is obvious that the API `connect` triggers the network fragment `e1 e2`, which includes the packets relevant to the task of establishing TCP handshake and SSL/TLS handshake. Then, Monkey tool publishes multiple messages to the broker. The invocation of `publish` triggers the network traffic `e3`. Then the `disconnect` method is invoked, which triggers the network traffic `e5` and ends the connection.

Fig.4(c) illustrates different behavior of the application events caused by the network traffic. To invoke `messageArrived`, we published multiple messages from Amazon Web Services (AWS) broker to the application. To ensure message delivery, we use QoS 1. After that, we connected the application to the broker. That invokes `connect` method and triggers `e1 e2`, followed by network traffic fragment `e4`, which is triggered by publishing the message from the broker. Then, `messageArrived` method is invoked. Because the invocation of `messageArrived` does not trigger any network traffic, the transition between `messageArrived` to `publish` or `disconnect` has no input.

5.3 NuSMV model and MQTT Security Verification

The conversion of the state machine to symbolic model generates that NuSMV file that contains 4 modules: main, Security, States, and Transitions. The following listings show the code snippet of main, Security, States, Transitions modules that model the state machine in Fig.4(d) for `MQTT Dashboard` application. The main module creates an instance of the other modules using the VAR declaration.

```
MODULE main
VAR
  security : Security;
  s        : States(n.NT);
  n        : Transitions(s.R);
```

The Security module defines the security properties of the application. These properties check whether the MQTT application validates the server X.509 certificate.

```
MODULE Security
DEFINE
  TLS           := TRUE;
  TLSv          := TRUE;
  certCheckCN   := TRUE;
  SignedCA      := TRUE;
  checkSigCorr  := TRUE;
  checkCAEXP    := TRUE;
```

The States module models the states of the MQTT-FSM of the application. Because of the `messageArrived` API invocation triggered by network traffic, we passed the network traffic NT as a parameter to this module.

```
MODULE States(NT)
VAR
R : {connect,publish,mesageArrived,disconnect,terminate};
ASSIGN
init(R) := connect;
next(R) := case
R = connect                    : {publish};
R = publish & NT = e3          : {publish, disconnect};
```

```
R = publish & NT = e3e4        : {mesageArrived};
R = mesageArrived & NT = e4    : {ma};
R = mesageArrived & NT = nop   : {publish};
R = disconnect                 : terminate;
TRUE                           : R;
esac;
```

The Transitions module models the transition of the MQTT-FSM of the application and takes API call as parameter.

```
MODULE Network(R)
VAR
NT : {e1e2,e3,e3e4,e4,e5,nop};
ASSIGN
init(NT) := e1e2;
next(NT) := case
R = connect     : e1e2;
R = publish     : {e3e4, e3};
R = ma          : {e4, nop};
R = disconnect  : e5;
TRUE            : NT;
esac;
```

After converting the MQTT-FSM to the symbolic model, the model checker of IoTVerif verifies both the general and security properties of the application. If MQTT client application provides secure connection over the latest version of SSL/TLS protocol and validates the certificate subjectCN, signature, and the certificate validity, the evaluation of all security specifications must all be *true*. Fig.5 illustrates the snippet of the specifications evaluation using the symbolic model above and the described specifications in Section 4.4 for `IoT MQTT Dashboard` application. It is clear that the application provides TLS and validates all the certificate components. Thus, all specifications are evaluated as *true*. However, Fig.6 illustrates the snippet of the specifications evaluation of `MQTT Viewer (IoT Client)` application. Although that application provides TLS, it does not validate the certificate , which exposes the application to MITM attack. Consequently, some security specifications, such as checking the certificate subjectCN and the signature, are evaluated as *false* and counter examples are given.

We have tested IoTVerif using 15 real-world Android MQTT applications. The detailed result of the tested 15 applications is presented in Table 4. We also include the statistics of network packets obtained from Monkey and the Paho API coverage. It is clear that the automation of network traces generation has high rates of API coverage for all apps: all are higher than 69%. We can notice from Table 4 that 5 (33.3%) of the applications are vulnerable to MITM. Only 1 application does not provide secure connection over SSL/TLS; hence, the application is vulnerable to various attacks. In addition, 3 applications provide secure connection over SSL/TLS, but they do not check the server certificate. Therefore, these 3 applications are vulnerable to MITM and TLS renegotiation attacks. Interestingly, 1 application provides secure connection and certificate validation, but the application uses TLSv1.0, which is vulnerable to various attacks as described in Section 2.2.

6 RELATED WORK

Verifying security properties of network protocols, including IoT protocols, is still undergoing research. Some works are closely related to our approach in terms of inferring a state machine of a protocol. De Ruiter and Poll [10] used automata algorithms to infer the TLS protocol model and inspected the state machine to find bugs and vulnerability. This work focused on the generated

```
-- specification AG TLS = TRUE IN security is true
-- specification AG TLSv >= 1 IN security is true
-- specification AG certCheckCN = TRUE IN security is true
-- specification AG SignedCA = TRUE IN security is true
-- specification AG checkSigCorr = TRUE IN security is true
-- specification AG certCheck = TRUE IN security is true
-- specification AG(! NT = RST AU R = disconnect) IN s is true
-- specification AG(R = connect -> AF (R = connect AU R = disconnect))
IN s is true
-- specification AG(NT = e2 -> (TLS ∧TLSv >= 1 & SignedCA &
certCheckCN & checkCAEXP)) IN security is true
                          .
                          .
```

Figure 5: The result of IoTVerif model checker of Android application, namely, `MQTT Dashboard`

```
-- specification AG TLS = TRUE IN security is true
-- specification AG TLSv >= 1 IN security is true
-- specification AG (certCheckCN = TRUE) IN secure is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
  -> State: 1.1 <-
    s.R = connect
    n.NT = e1
    security.TLS = TRUE
    security.TLSv = 1
    security.certCheckCN = FALSE
    secure.SignedCA = FALSE
    secure.checkSigCorr = FALSE
    secure.checkCAEXP = FALSE
```

Figure 6: The result of IoTVerif model checker of Android application, namely, `MQTT Viewer (IoT Client)`

TLS state machine during the handshake. Leita *et al.* [19] attempted to infer protocol state machine from the network traffic based on rebuilding TCP flows. Wang *et al.* [27] described the model probabilistically as the state machine based on the network trace of a specific application. In contrast, our system is based on both network traffic and application-level events, which can produce more accurate results.

There have been several efforts to investigate SSL/TLS security problems. Fahl *et al.* [12] statically analyzed Android applications from Google Play to find applications, which allow all host names and trust all certificates, vulnerability to MITM attack. Similarly, Sounthiraraj *et al.* [24] and FireEye [15] searched for `TrustManager` and `HostnameVerifier` issues to find applications vulnerability to MITM attack. Montelibano and Dormann [20] used CERT Tapioca tool to dynamically analyzed Android applications to find SSL/TLS vulnerabilities. The scope of these works does not include the study of the used SSL/TLS versions in the applications and their impact on SSL/TLS security. In contrast, our approach identifies SSL/TLS vulnerabilities based on the existence of the server certificate validation and the used SSL/TLS version in the client application.

7 CONCLUSION

In this paper, we presented IoTVerif, an automated tool that learns and identifies secure vulnerabilities, with respect to the usage of SSL/TLS, of IoT applications that utilize MQTT messaging protocol. IoTVerif simply correlates the observed network trace of an IoT applications with its execution context. IoTVerif does not require prior knowledge of protocols and is still able to identify various security vulnerabilities in software and enumerate the possible exploits. IoTVerif can help MQTT Android client developers to identify security vulnerability related to SSL/TLS certificate validation before they distribute their application through any marketplace. Our analysis of the 15 most popular free applications from the Google

Table 4: The detailed result for the 15 tested Android applications

App Package Name	Step ❶	Step ❸				Step ❹		Step ❺				Step ❼	
	# instrumented Paho Methods	# Network Packets by Monkey				Provide TLS?	TLS Version	Certificate Validation				vulnerable?	
		Min	Max	Average	Paho API Coverage Rate			Subject CN	Trusted Signature	Correct Signature	Certificate Validity	MITM	Renego
net.routix.mqttdash	1411	42	180	103	100%	✓	1.2	✓	✓	✓	✓	✗	✗
com.deepeshc.mqttrec	1245	25	117	71	82%	✓	1.2	✓	✓	✓	✓	✗	✗
com.patterns.io.mqttpatterns	1754	19	95	57	71%	✓	1.1	✓	✓	✓	✓	✗	✗
com.codifythings.universalmqttclient	621	25	125	75	88%	✓	1.0	✗	✗	✗	✗	✓	✓
br.com.bintechnology.mqttclient	785	27	135	81	78%	✗	N/A	✗	✗	✗	✗	✓	N/A
at.tripwire.mqtt.client	838	48	130	84	98%	✓	1.2	✓	✓	✓	✓	✗	✗
ru.esp8266.iotmanager	1656	27	119	73	75%	✓	1.2	✓	✓	✓	✓	✗	✗
com.thn.iotmqttdashboard	1083	58	200	119	71%	✓	1.2	✓	✓	✓	✓	✗	✗
com.vinmacro.vinmacromqtt	941	51	119	68	69%	✓	1.1	✗	✗	✗	✗	✓	✓
com.cmmakerclub.iot.cmmciotswitch	1768	23	115	69	84%	✓	1.1	✓	✓	✓	✓	✗	✗
info.laptrinhpic.easycontrol.mqttplugin	702	40	160	108	88%	✓	1.1	✓	✓	✓	✓	✗	✗
net.nosybore.mqttpublishplugin	1318	21	105	63	75%	✓	1.2	✓	✓	✓	✓	✗	✗
net.sabamiso.android.simplemqttviewer	758	44	110	75	88%	✓	1.0	✓	✓	✓	✓	✓	✓
in.dc297.mqttclpro	1226	27	135	81	72%	✓	1.1	✓	✓	✓	✓	✗	✗
com.hanzycanada.mqttspeed	1661	44	110	74.8	84%	✓	1.1	✗	✗	✗	✗	✓	✓

Play Market has shown that 5 applications are vulnerable to MITM and/or TLS Renegotiation attacks.

ACKNOWLEDGMENTS

This work is supported by the funding from National Science Foundation under award Grant No. DGE-1723707, DGE-1623713, and Michigan Space Grant Consortium.

REFERENCES

[1] Akdeniz. 2017. GitHub - Akdeniz/Google Play Crawler). https://github.com/Akdeniz/google-play-crawler. (2017).

[2] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mohammed Aledhari, and Moussa Ayyash. 2015. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys Tutorials* 17, 4 (Fourthquarter 2015), 2347–2376. https://doi.org/10.1109/COMST.2015.2444095

[3] Aisha Ali-Gombe, Irfan Ahmed, Golden G. Richard, III, and Vassil Roussev. 2016. AspectDroid: Android App Analysis System. In *Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy (CODASPY '16)*. ACM, New York, NY, USA, 145–147. https://doi.org/10.1145/2857705.2857739

[4] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*. ACM, New York, NY, USA, 259–269. https://doi.org/10.1145/2594291.2594299

[5] Microsoft Azure. 2017. Microsoft Azure IoT Hub. (2017). https://azure.microsoft.com/en-us/free/

[6] BetterCAP. 2017. MITM Tool and Framework. (2017). https://www.bettercap.org/

[7] Pavol Bielik, Veselin Raychev, and Martin Vechev. 2015. Scalable Race Detection for Android Applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2015)*. ACM, New York, NY, USA, 332–348. https://doi.org/10.1145/2814270.2814303

[8] Alessandro Cimatti, Edmund M. Clarke, Enrico Giunchiglia, Fausto Giunchiglia, Marco Pistore, Marco Roveri, Roberto Sebastiani, and Armando Tacchella. 2002. NuSMV 2: An OpenSource Tool for Symbolic Model Checking. In *Proceedings of the 14th International Conference on Computer Aided Verification (CAV '02)*. Springer-Verlag, London, UK, 359–364. http://dl.acm.org/citation.cfm?id=647771.734431

[9] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and Tim Polk. 2008. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. RFC Editor. http://www.rfc-editor.org/rfc/rfc5280.txt

[10] Joeri de Ruiter and Erik Poll. 2015. Protocol State Fuzzing of TLS Implementations. In *Proceedings of the 24th USENIX Security Symposium, USENIX Security 15, Washington, D.C., USA, August 12-14, 2015*. 193–206. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/de-ruiter

[11] Ernest Allen Emerson. 1990. Temporal and Modal Logic. MIT Press, Cambridge, MA, USA, Chapter Handbook of Theoretical Computer Science (Vol. B), 995–1072. http://dl.acm.org/citation.cfm?id=114891.114907

[12] Sascha Fahl, Marian Harbach, Thomas Muders, Lars Baumgärtner, Bernd Freisleben, and Matthew Smith. 2012. Why Eve and Mallory Love Android: An Analysis of Android SSL (in)Security. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12)*. ACM, New York, NY, USA, 50–61. https://doi.org/10.1145/2382196.2382205

[13] Simos Gerasimou, Radu Calinescu, and Alec Banks. 2014. Efficient Runtime Quantitative Verification Using Caching, Lookahead, and Nearly-optimal Reconfiguration. In *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2014)*. ACM, New York, NY, USA, 115–124. https://doi.org/10.1145/2593929.2593932

[14] HiveMq. 2017. MQTT Security Fundamentals: TLS/SSL. (2017). http://www.hivemq.com/blog/mqtt-security-fundamentals-tls-ssl

[15] FireEye Inc. 2014. SSL Vulnerabilities: Who listens when Android applications talk? (August 2014). https://www.fireeye.com/blog/threat-research/2014/08/ssl-vulnerabilities-who-listens-when-android-applications-talk.html

[16] IEEE IoT. 2016. IoT Developer Survey 2016 Final Report. (2016). https://iot.ieee.org/images/files/pdf/iot-developer-survey-2016-report-final.pdf

[17] Casper S. Jensen, Anders Møller, Veselin Raychev, Dimitar Dimitrov, and Martin Vechev. 2015. Stateless Model Checking of Event-driven Applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2015)*. ACM, New York, NY, USA, 57–73. https://doi.org/10.1145/2814270.2814282

[18] Ravi Kishore Kodali. 2016. An implementation of MQTT using CC3200. In *2016 International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT)*. 582–587. https://doi.org/10.1109/ICCICCT.2016.7988017

[19] Corrado Leita, Ken Mermoud, and Marc Dacier. 2005. ScriptGen: an automated script generation tool for Honeyd. In *21st Annual Computer Security Applications Conference (ACSAC'05)*. 12 pp.–214. https://doi.org/10.1109/CSAC.2005.49

[20] Joji Montelibano and Will Dormann. 2015. How We Discovered Thousands of Vulnerable Android Apps in One Day. https://www.rsaconference.com/events/us15/agenda/sessions/1638/how-we-discovered-thousands-of-vulnerable-android RSA Conference.

[21] Eric Rescorla, Marsh Ray, Steve Dispensa, and Nasko Oskov. 2010. *Transport Layer Security (TLS) Renegotiation Indication Extension*. RFC 5746. RFC Editor. http://www.rfc-editor.org/rfc/rfc5746.txt

[22] Yaron Sheffer, Ralph Holz, and Peter Saint-Andre. 2015. *Summarizing Known Attacks on Transport Layer Security (TLS) and Datagram TLS (DTLS)*. RFC 7457. RFC Editor. http://www.rfc-editor.org/rfc/rfc7457.txt

[23] Preeti Sirohi, Amit Agarwal, and Sapna Tyagi. 2016. A comprehensive study on security attacks on SSL/TLS protocol. In *Proceedings of the 2016 2nd International Conference on Next Generation Computing Technologies (NGCT)*. 893–898. https://doi.org/10.1109/NGCT.2016.7877537

[24] David Sounthiraraj, Justin Sahs, Garret Greenwood, Zhiqiang Lin, and Latifur Khan. 2014. Smv-hunter: Large scale, automated detection of ssl/tls man-in-the-middle vulnerabilities in android apps. In *Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS 14)*. Citeseer.

[25] Android Studio. 2017. UI/Application Exerciser Monkey | Android Studio. (2017). https://developer.android.com/studio/test/monkey.html

[26] Heila van der Merwe, Brink van der Merwe, and Willem Visser. 2014. Execution and Property Specifications for JPF-android. *SIGSOFT Softw. Eng. Notes* 39, 1 (Feb. 2014), 1–5. https://doi.org/10.1145/2557833.2560576

[27] Yipeng Wang, Zhibin Zhang, Danfeng Daphne Yao, Buyun Qu, and Li Guo. 2011. Inferring Protocol State Machine from Network Traces: A Probabilistic Approach. In *Proceedings of the 9th International Conference on Applied Cryptography and Network Security (ACNS'11)*. Springer-Verlag, Berlin, Heidelberg, 1–18.